

Библиотека за изчисления NumPy. Основни понятия. NumPy масиви. Изчисления с NumPy масиви

доц. д-р Гергана Спасова
Кат. „Компютърни науки и технологии“

Основни характеристики

- NumPy (Numerical Python) – една от най-важните библиотеки за обработване на данни
- Numpy - ефективна обработка на многомерни масиви
- Математически функции за бързо изпълнение на операции с целите масиви без необходимост от използване на оператори за цикъл
- Средства за четене/запис на данни от/на диска и работа с memory-mapped файлове
- Средства за линейна алгебра, генериране на случайни числа, и трансформации на Фурие
- API на C за свързване на NumPy с библиотеки реализирани на C, C++, FORTRAN

Основни приложения

- Бързо изпълнение на операции с векторизирани масиви за почистване, филтриране, обработване на данни
- Бързо сортиране на масиви, операции с множества
- Бързо агрегиране и обобщаване на данни
- Подравняване на данни, операции merge и join с релационни множества от данни
- Условни логически операции с масиви без цикли

Ефективност на NumPy при обработване на големи масиви от данни

- NumPy съхранява данни в последователни блокове от паметта. Функциите в NumPy са реализирани на С и обработват паметта бързо без допълнителни проверки (напр. type checking)
- NumPy масивите използват много по-малко памет в сравнение с вградените средства в Python
- Операциите в NumPy се изпълняват с целите масиви без необходимост от оператори за цикъл

Пример за ефективността на NumPy

In [7]: **import numpy as np**

In [8]: **my_arr = np.arange(10000000)**

In [9]: **my_list = list(range(10000000))**

- Умножение на елементите на двете множества по 2

In [10]: **%time for _ in range(10): my_arr2 = my_arr * 2**

CPU times: user 20 ms, sys: 50 ms, total: 70 ms

Wall time: 72.4 ms

In [11]: **%time for _ in range(10): my_list2 = [x * 2 for x in my_list]**

CPU times: user 760 ms, sys: 290 ms, total: 1.05 s

Wall time: 1.05 s

- NumPy базираните алгоритми са от 10 до 100 пъти (или повече) по-бързи в сравнение със стандартните средства в Python и използват много по-малко памет

Обект ndarray: многомерен масив

- Обект за работа с N-размерни масиви – позволява изпълнение на математически операции с големи множества от данни подобно на аналогичните операции със скаларни елементи

Пример:

```
In [12]: import numpy as np
```

```
# Generate some random data
```

```
In [13]: data = np.random.randn(2, 3)
```

```
In [14]: data
```

```
Out[14]:
```

```
array([[ -0.2047,  0.4789, -0.5194],  
       [-0.5557,  1.9658,  1.3934]])
```

Пример за операции с ndarray

In [15]: data * 10

Out[15]:

```
array([[ -2.0471,  4.7894, -5.1944],  
       [ -5.5573, 19.6578, 13.9341]])
```

In [16]: data + data

Out[16]:

```
array([[ -0.4094,  0.9579, -1.0389],  
       [ -1.1115,  3.9316,  2.7868]])
```


Ndarray

- Елементите в `ndarray` трябва да са от един тип
- За всеки обект `ndarray` е дефинирано свойство `shape`, което показва размерностите на масива
- За всеки обект `ndarray` е дефинирано свойство `dtype`, което показва типа на елементите на масива

```
In [17]: data.shape
```

```
Out[17]: (2, 3)
```

```
In [18]: data.dtype
```

```
Out[18]: dtype('float64')
```


Създаване на ndarray

- Функция `array` – извиква се с параметри друг масив или списък

Пример:

```
In [19]: data1 = [6, 7.5, 8, 0, 1]
```

```
In [20]: arr1 = np.array(data1)
```

```
In [21]: arr1
```

```
Out[21]: array([ 6. , 7.5, 8. , 0. , 1. ])
```

Конвертиране на вложени списъци в ndarray

```
In [22]: data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
```

```
In [23]: arr2 = np.array(data2)
```

```
In [24]: arr2
```

```
Out[24]:
```

```
array([[1, 2, 3, 4],  
       [5, 6, 7, 8]])
```

- Проверка на размерностите с `ndim` и `shape`

```
In [25]: arr2.ndim
```

```
Out[25]: 2
```

```
In [26]: arr2.shape
```

```
Out[26]: (2, 4)
```

Методи `zeros`, `ones` и `empty` за създаване на `ndarray`

```
In [29]: np.zeros(10)
```

```
Out[29]: array([ 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [30]: np.zeros((3, 6))
```

```
Out[30]:
```

```
array([[ 0., 0., 0., 0., 0., 0.],  
       [ 0., 0., 0., 0., 0., 0.],  
       [ 0., 0., 0., 0., 0., 0.]])
```

```
In [31]: np.empty((2, 3, 2))
```

```
Out[31]:
```

```
array([[[ 0., 0.],  
       [ 0., 0.],  
       [ 0., 0.]],  
       [[ 0., 0.],  
       [ 0., 0.],  
       [ 0., 0.]])
```


Функции за създаване на ndarray

Function	Description
<code>array</code>	Convert input data (list, tuple, array, or other sequence type) to an ndarray either by inferring a dtype or explicitly specifying a dtype; copies the input data by default
<code>asarray</code>	Convert input to ndarray, but do not copy if the input is already an ndarray
<code>arange</code>	Like the built-in <code>range</code> but returns an ndarray instead of a list
<code>ones</code> , <code>ones_like</code>	Produce an array of all 1s with the given shape and dtype; <code>ones_like</code> takes another array and produces a ones array of the same shape and dtype
<code>zeros</code> , <code>zeros_like</code>	Like <code>ones</code> and <code>ones_like</code> but producing arrays of 0s instead
<code>empty</code> , <code>empty_like</code>	Create new arrays by allocating new memory, but do not populate with any values like <code>ones</code> and <code>zeros</code>
<code>full</code> , <code>full_like</code>	Produce an array of the given shape and dtype with all values set to the indicated "fill value" <code>full_like</code> takes another array and produces a filled array of the same shape and dtype
<code>eye</code> , <code>identity</code>	Create a square $N \times N$ identity matrix (1s on the diagonal and 0s elsewhere)

Обект dtype

- Dtype е специален обект, съдържащ информация (метаданни) за типа на данните в ndarray

```
In [33]: arr1 = np.array([1, 2, 3], dtype=np.float64)
```

```
In [34]: arr2 = np.array([1, 2, 3], dtype=np.int32)
```

```
In [35]: arr1.dtype
```

```
Out[35]: dtype('float64')
```

```
In [36]: arr2.dtype
```

```
Out[36]: dtype('int32')
```


Типове данни в numpy

Type	Type code	Description
int8, uint8	i1, u1	Signed and unsigned 8-bit (1 byte) integer types
int16, uint16	i2, u2	Signed and unsigned 16-bit integer types
int32, uint32	i4, u4	Signed and unsigned 32-bit integer types
int64, uint64	i8, u8	Signed and unsigned 64-bit integer types
float16	f2	Half-precision floating point
float32	f4 or f	Standard single-precision floating point; compatible with C float
float64	f8 or d	Standard double-precision floating point; compatible with C double and Python float object
float128	f16 or g	Extended-precision floating point
complex64, complex128, complex256	c8, c16, c32	Complex numbers represented by two 32, 64, or 128 floats, respectively
bool	?	Boolean type storing True and False values
object	O	Python object type; a value can be any Python object
string_	S	Fixed-length ASCII string type (1 byte per character); for example, to create a string dtype with length 10, use 'S10'
unicode_	U	Fixed-length Unicode type (number of bytes platform specific); same specification semantics as string_ (e.g., 'U10')

Конвертиране на типове в ndarray

```
In [37]: arr = np.array([1, 2, 3, 4, 5])
```

```
In [38]: arr.dtype
```

```
Out[38]: dtype('int64')
```

```
In [39]: float_arr = arr.astype(np.float64)
```

```
In [40]: float_arr.dtype
```

```
Out[40]: dtype('float64')
```


Загуба на точност при конвертиране

```
In [41]: arr = np.array([3.7, -1.2, -2.6, 0.5, 12.9, 10.1])
```

```
In [42]: arr
```

```
Out[42]: array([ 3.7, -1.2, -2.6, 0.5, 12.9, 10.1])
```

```
In [43]: arr.astype(np.int32)
```

```
Out[43]: array([ 3, -1, -2, 0, 12, 10], dtype=int32)
```

Конвертиране на символни низови в числени стойности

```
In [44]: numeric_strings = np.array(['1.25', '-9.6', '42'], dtype=np.string_)
```

```
In [45]: numeric_strings.astype(float)
```

```
Out[45]: array([ 1.25, -9.6 , 42. ])
```

Аритметични операции с масиви в NumPy

- NumPy предоставя средства за операции с цели масиви без оператори за цикъл – **векторизация**
- Всяка аритметична операция с масиви с еднакви размерности се изпълнява поелементно

```
In [51]: arr = np.array([[1., 2., 3.], [4., 5., 6.]])
```

```
In [52]: arr
```

```
Out[52]:
```

```
array([[ 1.,  2.,  3.],  
       [ 4.,  5.,  6.]])
```

```
In [53]: arr * arr
```

```
Out[53]:
```

```
array([[ 1.,  4.,  9.],  
       [16., 25., 36.]])
```

```
In [54]: arr - arr
```

```
Out[54]:
```

```
array([[ 0.,  0.,  0.],  
       [ 0.,  0.,  0.]])
```


Арифметични операции с масив и скалярна стойност

In [55]: 1 / arr

Out[55]:

```
array([[ 1. , 0.5 , 0.3333],  
       [ 0.25 , 0.2 , 0.1667]])
```

In [56]: arr ** 0.5

Out[56]:

```
array([[ 1. , 1.4142, 1.7321],  
       [ 2. , 2.2361, 2.4495]])
```

Сравнение на масиви с еднакви размерности

```
In [57]: arr2 = np.array([[0., 4., 1.], [7., 2., 12.]])
```

```
In [58]: arr2
```

```
Out[58]:
```

```
array([[ 0.,  4.,  1.],  
       [ 7.,  2., 12.]])
```

```
In [59]: arr2 > arr
```

```
Out[59]:
```

```
array([[False,  True, False],  
       [ True, False,  True]], dtype=bool)
```

Индексиране на масиви

- Едномерните масиви се обработват като списък, но направените промени се отразяват в масива (не в негово копие)

Пример:

```
In [60]: arr = np.arange(10)
```

```
In [61]: arr
```

```
Out[61]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [62]: arr[5]
```

```
Out[62]: 5
```

```
In [63]: arr[5:8]
```

```
Out[63]: array([5, 6, 7])
```

```
In [64]: arr[5:8] = 12
```

```
In [65]: arr
```

```
Out[65]: array([ 0, 1, 2, 3, 4, 12, 12, 12, 8, 9])
```

Изрязване на масиви

```
In [66]: arr_slice = arr[5:8]
```

```
In [67]: arr_slice
```

```
Out[67]: array([12, 12, 12])
```

```
In [68]: arr_slice[1] = 12345
```

```
In [69]: arr
```

```
Out[69]: array([ 0, 1, 2, 3, 4, 12, 12345, 12, 8, 9])
```

```
In [70]: arr_slice[:] = 64
```

```
In [71]: arr
```

```
Out[71]: array([ 0, 1, 2, 3, 4, 64, 64, 64, 8, 9])
```


Обработване на многомерни масиви

- Елементите от всяка размерност представляват едномерни масиви (вектори)

Пример:

In [72]: `arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`

In [73]: `arr2d[2]`

Out[73]: `array([7, 8, 9])`

		axis 1		
		0	1	2
axis 0	0	0,0	0,1	0,2
	1	1,0	1,1	1,2
	2	2,0	2,1	2,2

Универсални функции

- Универсална функция или `ufunc` е функция, която изпълнява поелементни операции в `ndarray` масиви
- Универсалните функции са обвивки на опростени функции, които приемат като параметър една или повече скаларни стойности и връщат един или повече скаларни резултати

Унарни ufuncs - пример

```
In [137]: arr = np.arange(10)
```

```
In [138]: arr
```

```
Out[138]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [139]: np.sqrt(arr)
```

```
Out[139]:
```

```
array([ 0. , 1. , 1.4142, 1.7321, 2. , 2.2361, 2.4495, 2.6458, 2.8284, 3. ])
```

```
In [140]: np.exp(arr)
```

```
Out[140]:
```

```
array([ 1. , 2.7183, 7.3891, 20.0855, 54.5982, 148.4132, 403.4288, 1096.6332,  
2980.958 , 8103.0839])
```

Бинарни ufuncs - пример

```
In [141]: x = np.random.randn(8)
```

```
In [142]: y = np.random.randn(8)
```

```
In [143]: x
```

```
Out[143]:
```

```
array([-0.0119, 1.0048, 1.3272, -0.9193, -1.5491, 0.0222, 0.7584, -0.6605])
```

```
In [144]: y
```

```
Out[144]:
```

```
array([ 0.8626, -0.01 , 0.05 , 0.6702, 0.853 , -0.9559, -0.0235, -2.3042])
```

```
In [145]: np.maximum(x, y)
```

```
Out[145]:
```

```
array([ 0.8626, 1.0048, 1.3272, 0.6702, 0.853 , 0.0222, 0.7584, -0.6605])
```


Връщане на множество масиви от ufunc - пример

```
In [146]: arr = np.random.randn(7) * 5
```

```
In [147]: arr
```

```
Out[147]: array([-3.2623, -6.0915, -6.663 , 5.3731, 3.6182, 3.45 , 5.0077])
```

```
In [148]: remainder, whole_part = np.modf(arr)
```

```
In [149]: remainder
```

```
Out[149]: array([-0.2623, -0.0915, -0.663 , 0.3731, 0.6182, 0.45 , 0.0077])
```

```
In [150]: whole_part
```

```
Out[150]: array([-3., -6., -6., 5., 3., 3., 5.])
```

Списък с унарни ufuncs

Function	Description
<code>abs</code> , <code>fabs</code>	Compute the absolute value element-wise for integer, floating-point, or complex values
<code>sqrt</code>	Compute the square root of each element (equivalent to <code>arr ** 0.5</code>)
<code>square</code>	Compute the square of each element (equivalent to <code>arr ** 2</code>)
<code>exp</code>	Compute the exponent e^x of each element
<code>log</code> , <code>log10</code> , <code>log2</code> , <code>log1p</code>	Natural logarithm (base e), log base 10, log base 2, and $\log(1 + x)$, respectively
<code>sign</code>	Compute the sign of each element: 1 (positive), 0 (zero), or -1 (negative)
<code>ceil</code>	Compute the ceiling of each element (i.e., the smallest integer greater than or equal to that number)
<code>floor</code>	Compute the floor of each element (i.e., the largest integer less than or equal to each element)
<code>rint</code>	Round elements to the nearest integer, preserving the dtype
<code>modf</code>	Return fractional and integral parts of array as a separate array
<code>isnan</code>	Return boolean array indicating whether each value is NaN (Not a Number)
<code>isfinite</code> , <code>isinf</code>	Return boolean array indicating whether each element is finite (non- <code>inf</code> , non-NaN) or infinite, respectively
<code>cos</code> , <code>cosh</code> , <code>sin</code> , <code>sinh</code> , <code>tan</code> , <code>tanh</code>	Regular and hyperbolic trigonometric functions
<code>arccos</code> , <code>arccosh</code> , <code>arcsin</code> , <code>arcsinh</code> , <code>arctan</code> , <code>arctanh</code>	Inverse trigonometric functions
<code>logical_not</code>	Compute truth value of not x element-wise (equivalent to <code>~arr</code>).

Списък с бинарни ufuncs

Function	Description
<code>add</code>	Add corresponding elements in arrays
<code>subtract</code>	Subtract elements in second array from first array
<code>multiply</code>	Multiply array elements
<code>divide, floor_divide</code>	Divide or floor divide (truncating the remainder)
<code>power</code>	Raise elements in first array to powers indicated in second array
<code>maximum, fmax</code>	Element-wise maximum; <code>fmax</code> ignores NaN
<code>minimum, fmin</code>	Element-wise minimum; <code>fmin</code> ignores NaN
<code>mod</code>	Element-wise modulus (remainder of division)
<code>copysign</code>	Copy sign of values in second argument to values in first argument

Векторизация - пример

Пример: $\sqrt{x^2 + y^2}$

In [155]: `points = np.arange(-5, 5, 0.01) # 1000 equally spaced points`

In [156]: `xs, ys = np.meshgrid(points, points)`

In [157]: `ys`

Out[157]:

```
array([[ -5. , -5. , -5. , ..., -5. , -5. , -5. ],  
       [ -4.99, -4.99, -4.99, ..., -4.99, -4.99, -4.99],  
       [ -4.98, -4.98, -4.98, ..., -4.98, -4.98, -4.98],  
       ...,  
       [  4.97,  4.97,  4.97, ...,  4.97,  4.97,  4.97],  
       [  4.98,  4.98,  4.98, ...,  4.98,  4.98,  4.98],  
       [  4.99,  4.99,  4.99, ...,  4.99,  4.99,  4.99]])
```


Векторизация – пример (продължение)

```
In [158]: z = np.sqrt(xs ** 2 + ys ** 2)
```

```
In [159]: z
```

```
Out[159]:
```

```
array([[ 7.0711,  7.064 ,  7.0569, ...,  7.0499,  7.0569,  7.064 ],
       [ 7.064 ,  7.0569,  7.0499, ...,  7.0428,  7.0499,  7.0569],
       [ 7.0569,  7.0499,  7.0428, ...,  7.0357,  7.0428,  7.0499],
       ...,
       [ 7.0499,  7.0428,  7.0357, ...,  7.0286,  7.0357,  7.0428],
       [ 7.0569,  7.0499,  7.0428, ...,  7.0357,  7.0428,  7.0499],
       [ 7.064 ,  7.0569,  7.0499, ...,  7.0428,  7.0499,  7.0569]])
```

Векторизация – пример (продължение)

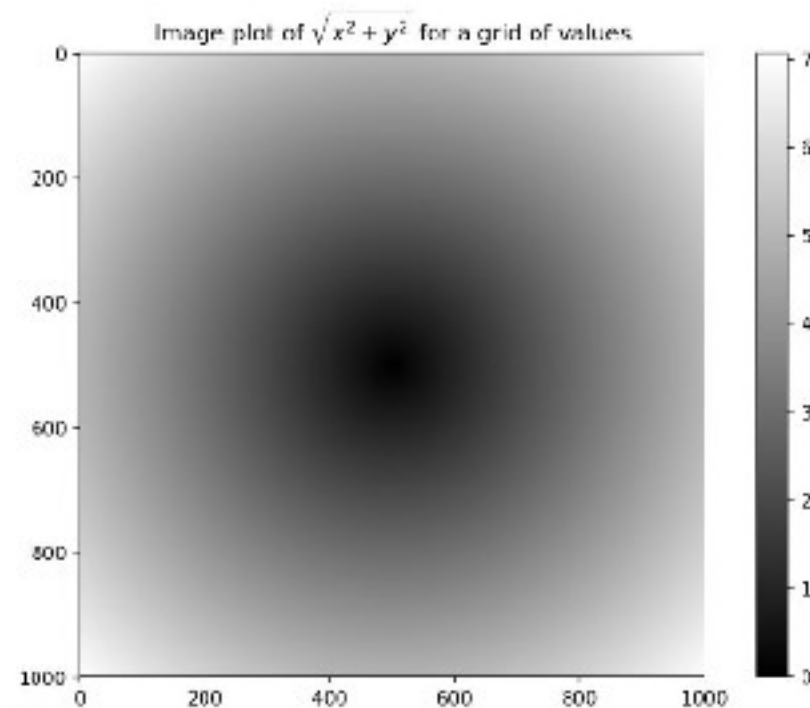
In [160]: **import matplotlib.pyplot as plt**

In [161]: plt.imshow(z, cmap=plt.cm.gray); plt.colorbar()

Out[161]: <matplotlib.colorbar.Colorbar at 0x7f715e3fa630>

In [162]: plt.title("Image plot of $\sqrt{x^2 + y^2}$ for a grid of values")

Out[162]: <matplotlib.text.Text at 0x7f715d2de748>



Векторизация на условни логически операции

- Функцията `numpy.where` е векторизирана версия на условния оператор `x if condition else y`

- Пример

```
In [165]: xarr = np.array([1.1, 1.2, 1.3, 1.4, 1.5])
```

```
In [166]: yarr = np.array([2.1, 2.2, 2.3, 2.4, 2.5])
```

```
In [167]: cond = np.array([True, False, True, True, False])
```

- Искаме да получим стойност от масива `xarr`, ако съответната стойност в `cond` е `True` или от масива `yarr` в противен случай

Реализация без векторизация

```
In [168]: result = [(x if c else y)
```

```
.....: for x, y, c in zip(xarr, yarr, cond)]
```

```
In [169]: result
```

```
Out[169]: [1.10000000000000000001, 2.20000000000000000002, 1.3,  
1.39999999999999999999, 2.5]
```

- Недостатъци на решението:
 - Бавно изпълнение за големи масиви (защото ще се изпълнява интерпретируем код)
 - Неприложимо за многомерни масиви

Реализация с векторизация

```
In [170]: result = np.where(cond, xarr, yarr)
```

```
In [171]: result
```

```
Out[171]: array([ 1.1, 2.2, 1.3, 1.4, 2.5])
```

Математически и статистически методи в NumPy

- NumPy съдържа методи на класа `array` за изпълнение на статистически операции с цял масив или с една от размерностите на масива
- Често използвани са методите `sum`, `mean` и `std`

Пример:

```
In [177]: arr = np.random.randn(5, 4)
```

```
In [178]: arr
```

```
Out[178]:
```

```
array([[ 2.1695, -0.1149, 2.0037, 0.0296],  
       [ 0.7953, 0.1181, -0.7485, 0.585 ],  
       [ 0.1527, -1.5657, -0.5625, -0.0327],  
       [-0.929 , -0.4826, -0.0363, 1.0954],  
       [ 0.9809, -0.5895, 1.5817, -0.5287]])
```

```
In [179]: arr.mean()
```

```
Out[179]: 0.19607051119998253
```

```
In [180]: np.mean(arr)
```

```
Out[180]: 0.19607051119998253
```

```
In [181]: arr.sum()
```

```
Out[181]: 3.9214102239996507
```

Други статистически методи

- `cumsum` И `cumprod` – изчисляват междинни суми и произведения в масива

```
In [184]: arr = np.array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
In [185]: arr.cumsum()
```

```
Out[185]: array([ 0, 1, 3, 6, 10, 15, 21, 28])
```

Cumsum и cumprod в многомерен масив

Пример:

```
In [186]: arr = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
```

```
In [187]: arr
```

```
Out[187]:
```

```
array([[0, 1, 2],  
       [3, 4, 5],  
       [6, 7, 8]])
```

```
In [188]: arr.cumsum(axis=0)
```

```
Out[188]:
```

```
array([[ 0,  1,  2],  
       [ 3,  5,  7],  
       [ 9, 12, 15]])
```

```
In [189]: arr.cumprod(axis=1)
```

```
Out[189]:
```

```
array([[ 0,  0,  0],  
       [ 3, 12, 60],  
       [ 6, 42, 336]])
```


Методи за работа с булеви масиви

- Булевите стойности се представят с 1 (True) и 0 (False). Методът `sum` често се използва за преброяване на стойностите `True` в булев масив

Пример:

```
In [190]: arr = np.random.randn(100)
```

```
In [191]: (arr > 0).sum() # Number of positive values
```

```
Out[191]: 42
```

Методи any и all

- any проверява дали една или повече стойности в масива са True, а all проверява дали всички стойности са True

Пример:

```
In [192]: bools = np.array([False, False, True, False])
```

```
In [193]: bools.any()
```

```
Out[193]: True
```

```
In [194]: bools.all()
```

```
Out[194]: False
```

- Методите any и all работят и с небулеви масиви, като ненулевите елементи се интерпретират със стойност True